Compiler Design

Assignment 2

Parser for C^-

Student: Mai Mahmoud Sayed

ID: 900213311

Course: CSCE 4101 – Compiler Design

Professor: Ahmed Rafea

Semester: Spring 2026

Contents

1 Introduction **2**

2 Grammar **2**

3 System Architecture **2**

4 Scanner Design **2**

5 Parser Design and Implementation Issues **3**
5.1 Left-Recursion Elimination 3
5.2 Corrected Statement-List Rule (Rule 5) 4
5.3 Dispatch on Lookahead 4
5.4 Dangling-Else Resolution 4
5.5 Declaration vs. Statement Disambiguation 4

6 Syntax Error Handling **5**

7 Sample Test Runs **5**
7.1 Test 1 – Valid: Arithmetic, If-Else, While 5
7.2 Test 2 – Valid: Arrays, Nested If, Relational Operators 6
7.3 Test 3 – Error: Missing Opening Brace 6
7.4 Test 4 – Error: Incomplete Expression 6
7.5 Test 5 – Error: Missing Assignment Operator 7
7.6 Test 6 – Error: Empty Statement-List (Corrected Rule 5) 7

8 Build Instructions **8**

9 Conclusion **8**

1 Introduction

This report documents the design and implementation of a recursive-descent parser for the C^- (C Minus) programming language, as required by Assignment 2 of CSCE 4101 Compiler Design. The parser accepts C^- source programs, validates their syntactic structure against the assignment grammar, and either reports a detailed syntax error (with line number and offending token) or prints a success message.

The scanner (lexical analyser) developed in Assignment 1 is reused as the token source. The two components together form the front-end of a C^- compiler.

Grammar Correction (Professor's Announcement): Rule 5 in the original assignment sheet contained a typo. The original rule:

`statement-list` → `statement-list ; statement | empty`

was corrected to:

`statement-list` → `statement-list ; statement | statement`

This change means every `statement-list` must contain *at least one* statement. Programs with an empty body are now a syntax error. The implementation reflects this corrected rule.

2 Grammar

Table 1 lists all 18 grammar rules as used in the implementation. Rules 2, 14, and 16 contained direct left recursion, which recursive-descent parsers cannot handle directly; these were transformed into iterative loops as described in Section 5.1.

3 System Architecture

The project is divided into four source files:

File	Responsibility
`tokens.h`	`TokenType` enum, `Token` struct, `token_type_str()` prototype
`scanner.h/.c`	Lexical analyser: character I/O, comment skipping, tokenisation
`parser.h/.c`	Recursive-descent parser: one function per non-terminal
`main.c`	Opens source file, initialises scanner, calls `parse_program()`

Token flow: `main` → `parse_program` → parser functions → `next_token` / `peek_token` → scanner.

4 Scanner Design

The scanner maintains a one-character pushback buffer (`ugc`/`gc`) that enables two-character operators (`<=`, `>=`, `==`, `!=`) to be recognised: the first character is read, then the next character is peeked; if it completes the operator the pair is consumed, otherwise the second character is pushed back.

Table 1: C⁻ Grammar (corrected per professor's announcement)

Rule	Non-terminal	Production
1	*program*	`Program ID { declaration-list statement-list }`
2	*declaration-list*	`var-declaration { var-declaration }` (LR eliminated)
3	*var-declaration*	`type-specifier ID ;` | `type-specifier ID [ NUM ] ;`
4	*type-specifier*	`int` | `float`
5	*statement-list*	`statement { ; statement }` (LR eliminated, corrected)
6	*statement*	*assignment-stmt* | *compound-stmt* | *selection-stmt* | *iteration-stmt*
7	*assignment-stmt*	`var = expression`
8	*var*	`ID` | `ID [ expression ]`
9	*compound-stmt*	`{ statement-list }`
10	*selection-stmt*	`if ( expression ) statement` | `if ( expression ) statement else statement`
11	*iteration-stmt*	`while ( expression ) statement`
12	*expression*	*additive-expression relop additive-expression* | *additive-expression*
13	*relop*	`<=` | `<` | `>` | `>=` | `==` | `!=`
14	*additive-expression*	`term { addop term }` (LR eliminated)
15	*addop*	`+` | `-`
16	*term*	`factor { mulop factor }` (LR eliminated)
17	*mulop*	`*` | `/`
18	*factor*	`( expression )` | *var* | `NUM`

Block comments (`/* ... */`) are consumed transparently inside the scanner loop, so the parser never sees comment tokens.

The public interface provides two functions:

- `next_token()` – consumes and returns the next token.
- `peek_token()` – returns the next token *without* consuming it, using a one-token buffer. This is used by `is_statement_first()` and similar predicates.

5 Parser Design and Implementation Issues

5.1 Left-Recursion Elimination

Three grammar rules had direct left recursion and were rewritten as iterative loops, as recursive-descent parsers would loop infinitely otherwise:

1. **Rule 2** (*declaration-list*): `parse_declaration_list()` calls `parse_var_declaration()` at least once, then loops while the lookahead is a type specifier (`int` or `float`).
2. **Rule 14** (*additive-expression*): `parse_additive_expression()` parses one *term*, then loops while the lookahead is `+` or `-`.

3. **Rule 16** (*term*): `parse_term()` parses one *factor*, then loops while the lookahead is `*` or `/`.

5.2 Corrected Statement-List Rule (Rule 5)

The corrected Rule 5 eliminates left recursion and removes the `empty` alternative, resulting in the implementation below:

```
static void parse_statement_list(void) {
    parse_statement();            /* at least one statement required */

    while (lookahead.type == TK_SEMI) {
        advance();                /* consume ';' separator */
        if (!is_statement_first()) break; /* tolerate trailing ';' */
        parse_statement();
    }
}
```

Listing 1: parse_statement_list – corrected rule implementation

The semicolon is a *separator* between statements, not a terminator. A trailing `;` after the last statement is consumed gracefully. Because the `empty` alternative was removed, every statement-list must contain at least one statement – empty compound bodies `{ }` are now a syntax error.

5.3 Dispatch on Lookahead

Every parsing decision is driven by the single lookahead token:

- `parse_statement()` switches on the lookahead: `ID` → assignment-stmt; `{` → compound-stmt; `if` → selection-stmt; `while` → iteration-stmt.
- `parse_factor()` dispatches on: `(` → parenthesised expression; `ID` → variable; `NUM` → number literal.
- `parse_expression()` checks for a relational operator after the first additive-expression; if none is present it returns normally (single additive-expression case, Rule 12 second alternative).

5.4 Dangling-Else Resolution

The if–else ambiguity is resolved greedily. After parsing the then-branch, `parse_selection_stmt()` immediately checks whether the lookahead is `else` and, if so, consumes it and parses the else-branch. This correctly associates each `else` with the nearest unmatched `if`, which is the standard convention in C-like languages.

5.5 Declaration vs. Statement Disambiguation

Declarations always start with `int` or `float`, while statements start with `ID`, `{`, `if`, or `while`. These FIRST sets are disjoint, so `parse_declaration_list()` and `parse_statement_list()` can be called sequentially with no ambiguity.

6 Syntax Error Handling

When an unexpected token is encountered, the `syntax_error()` function:

1. Prints the **line number** of the offending token.
2. Prints the **lexeme** (actual text) of the lookahead token.
3. Prints a human-readable description of the **expected** token or construct.
4. Calls `exit(1)` to terminate compilation immediately.

Error recovery is not implemented, as specified by the assignment. The `match()` helper centralises the error message format so all mismatch errors are consistent. Individual parse functions also call `syntax_error()` with custom expected-strings when no alternative matches (e.g. inside `parse_factor()` or `parse_type_specifier()`).

Error message format:

```
Syntax error at line <N>: unexpected token "<lexeme>", expected <
   description>
```

7 Sample Test Runs

Six test cases were prepared: two valid programs and four erroneous programs. Test 6 is specifically designed to demonstrate the behaviour difference introduced by the corrected Rule 5.

7.1 Test 1 – Valid: Arithmetic, If-Else, While

```
Program test1 {
    int x;
    int y;
    float result;

    x = 10;
    y = 3;
    result = 0.0;

    while (x > 0) {
        x = x - 1;
        if (x == y) {
            result = x + y
        } else {
            result = x * y
        }
    };

    y = (x + 5) * 2
}
```

Listing 2: test1.cm – valid program

```
$ ./cparser test1.cm
Parsing completed successfully.
```

7.2 Test 2 – Valid: Arrays, Nested If, Relational Operators

```
Program test2 {
    int arr[10];
    int i;
    int total;
    float avg;

    i = 0;
    total = 0;

    while (i < 10) {
        arr[i] = i * 2;
        total = total + arr[i];
        i = i + 1
    };

    avg = total / 10;

    if (avg >= 5) {
        if (avg != 9) {
            total = total - 1
        }
    } else {
        total = 0
    }
}
```

Listing 3: test2.cm – valid program with arrays

```
$ ./cparser test2.cm
Parsing completed successfully.
```

7.3 Test 3 – Error: Missing Opening Brace

The `{` after the program name is absent. The parser expects `{` after the identifier but finds `int`.

```
Program test3
    int x;
    x = 5
}
```

Listing 4: test3_err.cm – missing opening brace

```
$ ./cparser test3_err.cm
Syntax error at line 3: unexpected token "int", expected '{'
```

7.4 Test 4 – Error: Incomplete Expression

The expression `y = x +` is missing its right-hand operand. `parse_factor()` encounters `}` which is not a valid factor start.

```
Program test4 {
    int x;
    int y;

    x = 10;
    y = x +
}
```

Listing 5: test4_err.cm – missing right-hand operand

```
$ ./cparser test4_err.cm
Syntax error at line 8: unexpected token "}", expected a factor:
    '(' expression ')', variable name, or number
```

7.5 Test 5 – Error: Missing Assignment Operator

Statement `b 10` is missing the `=`. After consuming `b`, the parser expects `=` but finds `10`.

```
Program test5 {
    int a;
    int b;

    a = 5;
    b 10
}
```

Listing 6: test5_err.cm – missing assignment operator

```
$ ./cparser test5_err.cm
Syntax error at line 7: unexpected token "10", expected '='
```

7.6 Test 6 – Error: Empty Statement-List (Corrected Rule 5)

This test specifically targets the corrected Rule 5. Under the *original* (erroneous) rule, the `| empty` alternative would have made this program valid. Under the *corrected* rule, a statement-list must contain at least one statement, so the `}` immediately following the declaration triggers a syntax error.

```
/* Empty program body -- no statements present.
   Valid under old rule (| empty), invalid under corrected rule.
      */
Program test6 {
    int x;
}
```

Listing 7: test6_err.cm – empty body, valid under old rule only

```
$ ./cparser test6_err.cm
Syntax error at line 5: unexpected token "}", expected
    a statement: ID, '{', 'if', or 'while'
```

This confirms the corrected grammar is enforced by the implementation.

8 Build Instructions

Place all source files (`tokens.h`, `scanner.h`, `scanner.c`, `parser.h`, `parser.c`, `main.c`, `Makefile`) in the same directory, then run:

```
$ cd parser_project/
$ make
$ ./cparser <source_file.cm>
```

To remove compiled objects and the executable: `make clean`

9 Conclusion

The recursive-descent parser correctly validates C^- programs against all 18 grammar rules, including the professor's corrected Rule 5. Three left-recursive rules were transformed into iterative loops. The dangling-else ambiguity was resolved greedily. Syntax errors are reported with line number, offending token, and a description of what was expected, satisfying all assignment requirements.